

SE2020 – Web and Mobile Technologies
BSc (Hons) in Software Engineering
2nd Year
Faculty of Computing
SLIIT
2026 – Lab 09

Lab Sheet – React Basics

Objectives

At the end of this lab, students will be able to:

- Understand what React is
- Create a React application using Vite
- Understand project structure
- Create and use React components
- Use JSX syntax
- Pass data using props
- Handle basic events

Prerequisites

- Node.js 18+ installed
- Basic JavaScript knowledge
- A code editor (VS Code)

Task 1: Check Node.js Installation

1. Install node js to laptop

Link - <https://nodejs.org/en/download>

The screenshot shows the 'Download Node.js®' page. At the top, there are dropdown menus to select the version (v24.12.0 (LTS)), operating system (Windows), and package manager (npm). Below these is a green banner with the text: 'Info Want new features sooner? Get the latest Node.js version instead and try the latest improvements!'. The main content area displays a list of instructions for installing Node.js using Docker. The instructions are as follows:

```
1 # Docker has specific installation instructions for each operating system.
2 # Please refer to the official documentation at https://docker.com/get-started/
3
4 # Pull the Node.js Docker image:
5 docker pull node:24-alpine
6
7 # Create a Node.js container and start a Shell session:
8 docker run -it --rm --entrypoint sh node:24-alpine
9
10 # Verify the Node.js version:
11 node -v # Should print "v24.12.0".
12
13 # Verify npm version:
14 npm -v # Should print "11.6.2".
```

At the bottom of the instructions, there is a 'PowerShell' label and a 'Copy to clipboard' button.

2. Open Terminal / Command Prompt
3. Run the following command:

Windows OS - `node -v`

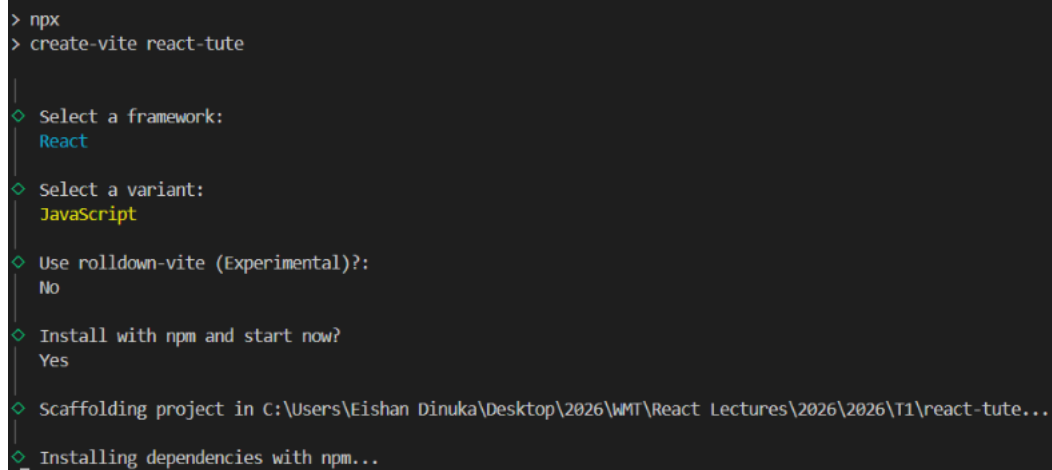
Mac OS - `node -v`

If Node.js is installed, the version number will be displayed.

Task2: Create React App Using Vite

1. Open Terminal
2. Run the following command

`npm create vite@4.0.0 react-basic-lab`



```
> npx
> create-vite react-tute

◇ Select a framework:
  React

◇ Select a variant:
  JavaScript

◇ Use rolldown-vite (Experimental)?:
  No

◇ Install with npm and start now?
  Yes

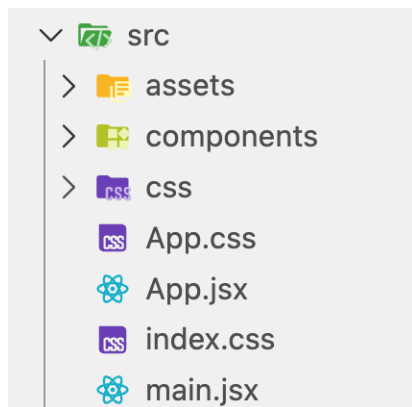
◇ Scaffolding project in C:\Users\Eishan Dinuka\Desktop\2026\WMT\React Lectures\2026\2026\T1\react-tute...

◇ Installing dependencies with npm...
```

3. Navigate to the folder
4. Create the project struct as follows.

`cd latest react-basic-lab`

Inside the 'scr' folder create the following 'components' folder.



Inside the component file create three files calls ' NavBar.jsx ', ' Pet.jsx ' and ' Cart.jsx '.

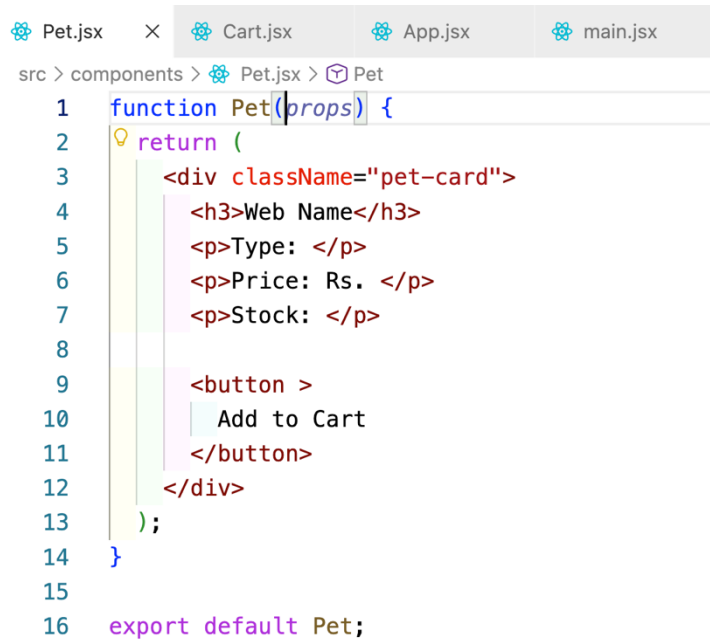
Inside the ' NavBar.jsx ' type this code.

```
Pet.jsx Cart.jsx App.jsx NavBar.jsx X
src > components > NavBar.jsx > Navbar
1 function Navbar(props) {
2
3   return (
4     <nav className="navbar">
5       <h2>🐾 Pet Shop</h2>
6
7       <ul className="nav-links">
8         <li>Home</li>
9         <li>Pets</li>
10        <li>Contact</li>
11      </ul>
12    </nav>
13  );
14 }
15
16 export default Navbar;
17
```

Then inside the ' Cart.jsx ' add this code.

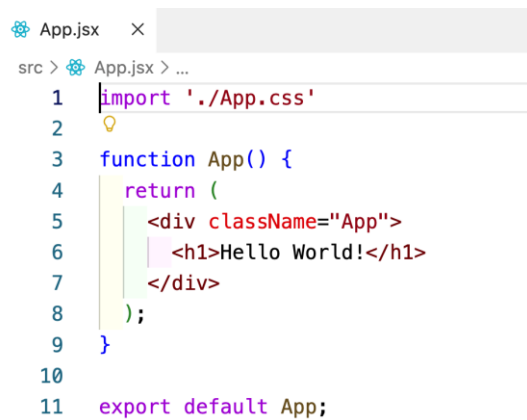
```
Pet.jsx Cart.jsx X App.jsx NavBar.jsx
src > components > Cart.jsx > Cart
1 function Cart() {
2
3   return (
4     <div className="cart">
5       <h2>🛒 Shopping Cart</h2>
6       <p>Total Items in Cart:</p>
7     </div>
8   );
9 }
10
11 export default Cart;
```

Finally add below code inside the ' Pet.jsx ' .



```
1 function Pet(props) {
2   return (
3     <div className="pet-card">
4       <h3>Web Name</h3>
5       <p>Type: </p>
6       <p>Price: Rs. </p>
7       <p>Stock: </p>
8
9       <button >
10        Add to Cart
11      </button>
12    </div>
13  );
14 }
15
16 export default Pet;
```

5. Replace all the content of the App.jsx file with the code below



```
1 import './App.css'
2
3 function App() {
4   return (
5     <div className="App">
6       <h1>Hello World!</h1>
7     </div>
8   );
9 }
10
11 export default App;
```

Task 3: Component

1. Inside the 'App.jsx' file call the 'Navbar' , "Pet" and 'Cart' file as follows and import it as below.
2. Then run and see the output.
3. Inside the 'App.jsx' file call the student file as follows and import it as below.

```

import NavBar from "./components/Navbar";
import Pet from "./components/Pet";
import Cart from "./components/Cart";

function App() {
  return (
    <div>
      <Navbar />
      <div className="container">
        <h1>🐾 Welcome to Pet Shop</h1>
        <p>Check pet availability and add to cart!</p>
        <Pet />
        <Pet />
        <Pet />
        <Cart />
      </div>
    </div>
  );
}

export default App;

```

4. Create folder called 'css' inside the 'src' folder. Inside that one create a file call 'styles.css'. Copy past the code which included inside the zip folder. 'styles.css'.

Task 4: Usage of props

1. Inside the NavBar.jsx add the below code

```
<li>🛒 Cart ({props.cartCount})</li>
```

Explanation:

- `props.cartCount` passes **cart count from App** to NavBar
- Props allow **data transfer from parent to child**

2. Inside the 'Pet.jsx' file add below code

```

Pet.jsx  X  Cart.jsx  App.jsx  main.jsx
src > components > Pet.jsx > Pet
1  function Pet() {
2      return (
3          <div className="pet-card">
4              <h3>Web Name</h3>
5              <p>Type: </p>
6              <p>Price: Rs. </p>
7              <p>Stock: </p>
8
9              <button >
10                 Add to Cart
11             </button>
12         </div>
13     );
14 }
15
16 export default Pet;

```

3. Add the below code inside the 'App.js' file

```

16     <Pet
17         name="Golden Retriever"
18         type="Dog"
19         price={50000}
20     />
21
22     <Pet
23         name="Persian Cat"
24         type="Cat"
25         price={35000}
26     />
27
28     <Pet
29         name="Parrot"
30         type="Bird"
31         price={15000}
32     />

```

Task 5: React Hooks

1. In the scenario now we are going to implement the **add to cart function** , **decrease the stock count** and the **cart count in the navigation bar**.

Firstly create the two use state .One for cart count other one for stock count.

At the initially we assign the cart count as zero. There for inside the useState bracket we input 0 . And the initial values for the stock we need to add a count like as follows.

```
const [cartCount, setCartCount] = useState(0); // useState for cart  
  
const [stock, setStock] = useState({ // useState for pet stock  
  "Golden Retriever": 5,  
  "Persian Cat": 3,  
  "Parrot": 10  
});
```

2. Then we need to create the function call addToCart () . Inside that function need to be included followings.

```
function addToCart(petName) { // Event handler  
  if (stock[petName] > 0) {  
    setCartCount(cartCount + 1);  
    setStock({ ...stock, [petName]: stock[petName] - 1 });  
  } else {  
    alert(`${petName} is out of stock!`);  
  }  
}
```

- b. Using useEffect() Hook

3. Add below code inside the 'App.jsx file'.

```
// useEffect for side effects  
useEffect(() => {  
  console.log("Cart count:", cartCount);  
  console.log("Current stock:", stock);  
}, [cartCount, stock]);
```

Inside the console see what happen.

Explanation: We use useEffect here to run extra code after the component updates, and what happens is that whenever the cart count or stock changes, React automatically runs this code and logs the updated values to the console, helping us track changes or perform side tasks like logging or API calls.

Task 6: Handling Events

We use the onClick event handler inside the button so that when the user clicks the button, React executes the given function (like adding a pet to the cart), allowing the app to respond to user actions.

```
<button onClick={props.onAddToCart} disabled={props.stock === 0}>  
  {props.stock > 0 ? "Add to Cart" : "Out of Stock"}  
</button>
```

Explanation: This button calls the add to cart function when clicked, and it automatically disables itself and shows “Out of Stock” when the stock becomes zero, otherwise it shows “Add to Cart”.

Final Output:

